Fault-Tolerant Totally-Ordered Multicast: Specification

1. Introduction

This specification defines the architectural requirements, formal behavioral contracts, and public programmatic interfaces for a middleware layer enabling fault-tolerant, totally-ordered atomic multicast. The system operates atop an asynchronous, non-blocking message-passing substrate (QueueBroker and MessageQueue abstractions) confined to a single Java Runtime Environment (JRE) process boundary.
The primary purpose of the middleware is to present an execution environment wherein concurrent actors coordinate actions through deterministic, reliable state-machine replication. By abstracting the complexities of dynamic group membership, message interleaving, and arbitrary crash failures, the protocol ensures that totally-ordered multicast semantics are maintained deterministically across all active participants.

2. Scope

This specification covers the behavioral semantics, message structures, failure handling, and public APIs of the Totally-Ordered Multicast middleware. It defines the rules for logical clock management, actor lifecycles, and verification procedures. It does not cover the internal implementation details of the underlying transport layer (QueueBroker), which is assumed to be a provided black-box abstraction.

3. Definitions and Terminology

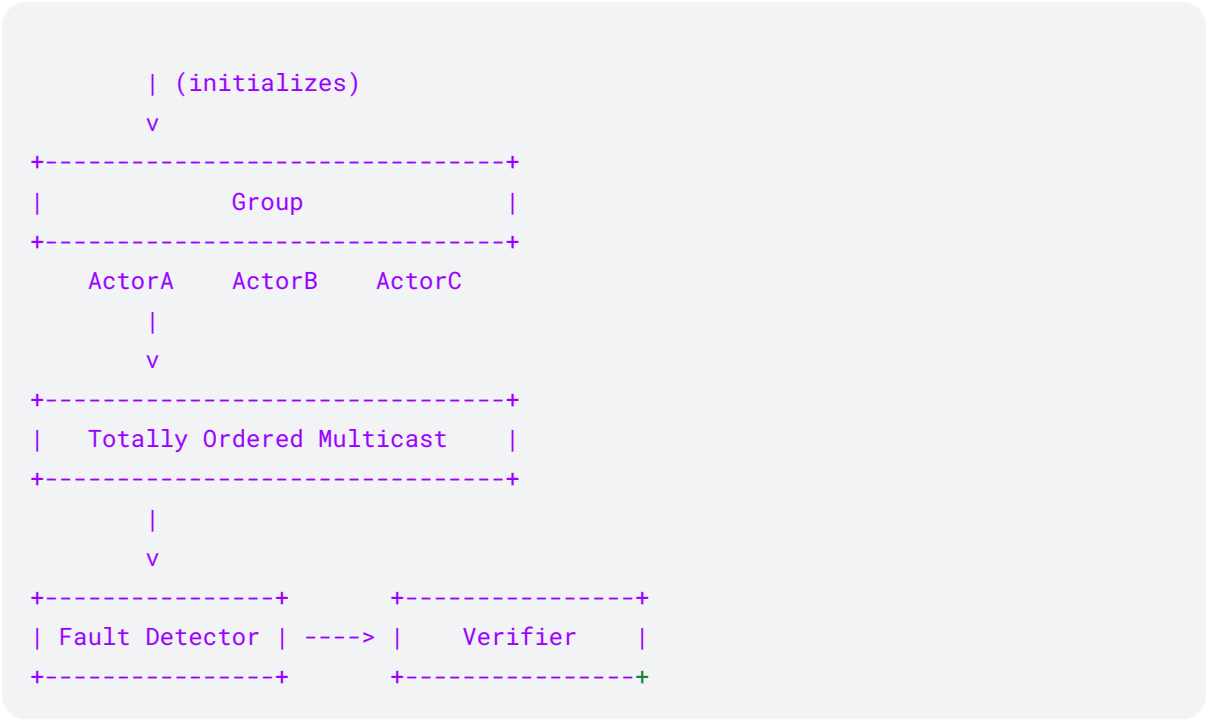
- **Actor**: An independent concurrent entity participating in the multicast group, encapsulating its own state and event-driven task.
- **View**: The set of active, correct actors currently recognized by the system.
- **Fail-Stop**: A failure model where an actor halts execution completely without Byzantine (malicious) behavior.
- **Generation ID**: A unique identifier marking the specific lifecycle epoch of an actor, preventing message bleed-over after a resurrection.
- **Multicast**: The act of broadcasting a message to all members of the current view.

4. System Overview

The system is composed of several decoupled interacting components ensuring separation of concerns between topology management, totally-ordered multicast, and verification.

4.1 Architecture Diagram

```
None
+----------------+
| Group Manager  |
+----------------+
```



4.2 Group Manager

A centralized bootstrap component responsible for establishing the initial static group of N actors. It allocates initial views, instantiates the Fault Detector, registers the Verifier, and then permanently departs the topology, leaving a pure peer-to-peer (P2P) group.

4.3 Actors

The core participants. Each actor manages a private execution queue, a logical clock, and a local message buffer. Actors communicate strictly via the asynchronous message substrate.

4.4 Fault Detector

A global service monitoring actor health. It notifies surviving actors of crashes, leaves, and joins to maintain view synchrony.

4.5 Verifier

An out-of-band monitoring singleton. It receives synchronous, zero-network notifications of every delivered message to deterministically validate that the total order is uniformly respected.

5. Functional Requirements

- **FR-1 Group Creation**: The system shall initialize a static group of N actors via a Group Manager.
- **FR-2 Message Communication**: Actors shall communicate exclusively via non-blocking MessageQueues.
- **FR-3 Totally Ordered Multicast**: The system shall deliver all multicast messages in the exact same sequence to all correct actors.

- **FR-4 Logical Clocks**: Every actor shall maintain a Lamport Logical Clock that ticks per execution event.
- **FR-5 Failure Detection**: The system shall provide perfect fault detection (Strong Completeness and Strong Accuracy).
- **FR-6 Actor Failures**: The system shall support both voluntary actor suicides and random canFail task crashes.
- **FR-7 Resurrection**: An actor shall be able to respawn with a clean state after a failure.
- **FR-8 Reinsertion**: A resurrected actor shall be seamlessly reintegrated into the active topology view.
- **FR-9 Dynamic Membership**: The system shall support dynamic joining and leaving of actors at runtime.
- **FR-10 Verification**: An external Verifier shall validate delivery invariants without altering network behavior.

6. System and Failure Model

6.1 Asynchronous Execution

- **Channel Characteristics**: The underlying transport layer models fully asynchronous message passing with FIFO delivery between pairs of actors, imposing no upper bounds on transmission delay.
- **Scheduling and Latency**: Thread dispatching and execution speeds are unbounded. No synchronized wall-clock times are assumed.

6.2 Failure Model

- **Crash-Stop Semantics**: Actors execute correctly according to their programmed transition functions until an unrecoverable crash event occurs.
- **Failure Modes**: Deterministic termination (suicide) or non-deterministic faults (stochastic failure via canFail).
- **Non-Byzantine Guarantee**: Failures are strictly benign; compromised internal states or adversarial messages are explicitly excluded.

6.3 Fault Detection

- **Detector Classification**: The subsystem relies on an idealized, perfect fault detector (Class P).
- **Strong Completeness**: Every crashed actor is eventually detected by every surviving correct actor.
- **Strong Accuracy**: No actor is suspected before it has actually crashed.

7. Group Formation and Membership

The topology evolves from a static managed state to a dynamic P2P mesh. Upon Group Manager bootstrap, all actors are fully meshed. As the system progresses, membership dynamically scales via explicit JOIN, LEAVE, and CRASH notifications propagated by the Fault Detector, requiring surviving actors to recalculate their membership view.

8. Message Model

To guarantee ordering and isolation, every transit payload is encapsulated in a formal structural envelope.

```java
Message {
    long messageId;         // Unique deterministic identifier for the message
    String senderId;        // The actor ID originating the message
    long generationId;      // The current lifecycle epoch of the sender
    long lamportTimestamp;  // The logical clock value at the time of sending
    byte[] payload;         // The opaque application data
}
```

9. Totally Ordered Multicast

9.1 Ordering Rule (Lamport)

Messages are strictly ordered across the distributed system using the Lamport happens-before timestamping algorithm. For any two messages m_1 and m_2, m_1 precedes m_2 ($m_1 < m_2$) if and only if:

1. $TS(m_1) < TS(m_2)$
2. OR, if $TS(m_1) == TS(m_2)$, then $ActorId(m_1) < ActorId(m_2)$ (Lexicographical tie-breaker).

9.2 Delivery Conditions

A message is dispatched to the application layer ONLY when it becomes globally stable. An actor p delivers m iff:

1. m is at the head of p's local priority queue.
2. A message becomes deliverable only when the multicast protocol can determine that its final global position is stable across the active View.

9.3 Multicast Sequence Diagram



```
   |                              |                                |
|
   |                              |--- deliver(m)
------------------------------->|
   |                              |                                |
|
```

10. Formal Delivery Guarantees

10.1 Uniform Agreement

If any actor p_i (whether correct or subsequently crashed) executes deliver(m), then every correct actor p⍰ must eventually execute deliver(m).

10.2 Integrity

An actor delivers m at most once, and only if some actor previously executed multicast(m).

10.3 Uniform Total Order

If actor p_i delivers m_1 before m_2, then no actor p⍰ delivers m_2 before m_1.

10.4 Isolation

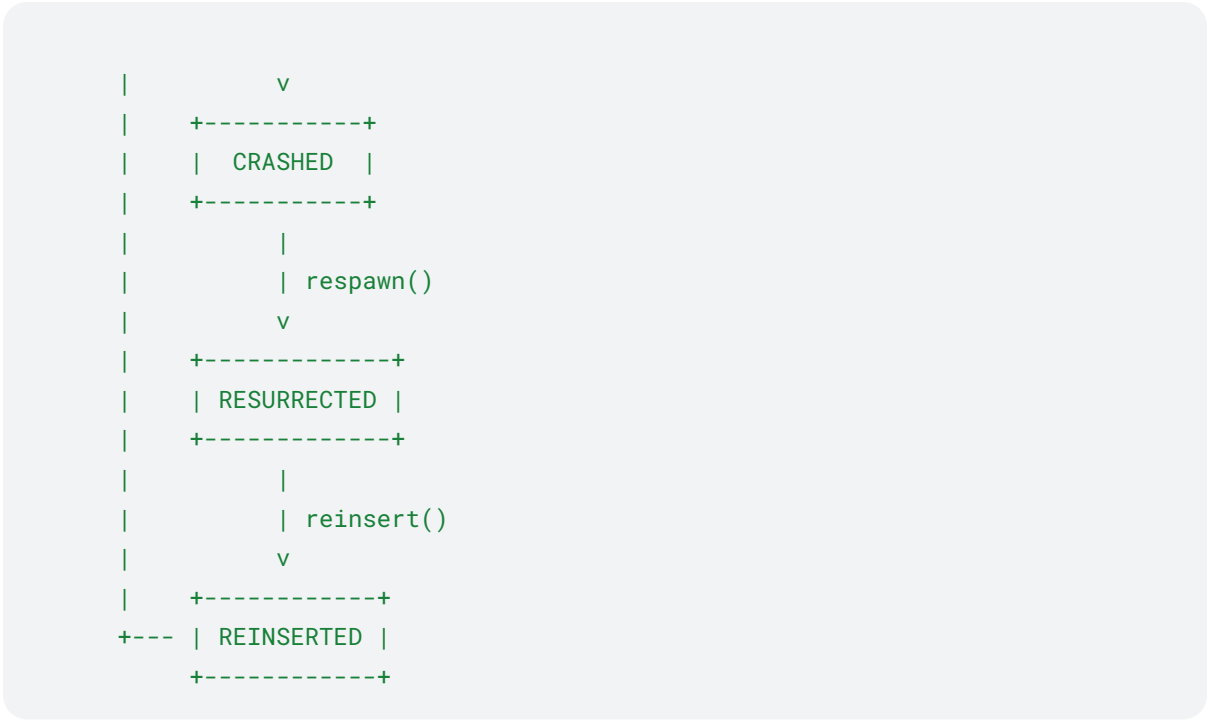
Any message m emitted by actor p⍰ after the fault detector establishes p⍰'s crash must be dropped.

11. Actor Lifecycle

Actors progress through a strict finite state machine transitioning their multicast semantics capabilities.

None

```
+-----------+
|  CREATED  |
+-----------+
      |
      | start()
      v
+-----------+          voluntary leave
|           | ----------------------------------> +-----------+
|  ACTIVE   |                                     |   LEFT    |
|           | ---+                                +-----------+
+-----------+    |
      ^          | crash / fault
```



12. Failure Recovery and Reinsertion

When an actor crashes, its memory and queues are irrevocably destroyed.

- **Resurrection**: If the actor respawns, it does so with an empty state and a new generationId.
- **No Retrospective Replay**: The system does NOT replay the historical message log. A resurrected actor begins participating from the exact moment of its reinsertion, permanently missing any messages delivered during its downtime.

13. Verifier Specification

13.1 Purpose

The Verifier acts as an oracle to validate the protocol's correctness. It asserts:

- Uniform Agreement (lists match)
- Integrity (no duplicates)
- Uniform Total Order (sequence identical)
- Correct Membership awareness
- **Missed-message validation**: The Verifier shall ensure that messages missed during actor crashes remain unavailable after reinsertion.

13.2 Observed Events

The Verifier exposes direct Java method calls to observe the following without adding network latency:

- deliveries
- crashes
- resurrections

- reinsertions
- joins
- leaves

14. Incremental Development Stages

1. **Stage 1**: Totally-ordered multicast without actor failures.
2. **Stage 2**: With actor failures, but no reinsertion (shrinking group).
3. **Stage 3**: With actor failures, resurrection, and reinsertion.
4. **Stage 4**: Fully dynamic group (voluntary join/leave and ungraceful crashes).

15. Testing Requirements

- **TR-1**: Validate Lamport Logical Clocks behavior under load.
- **TR-2**: Validate static Group formation and View distribution.
- **TR-3**: Execute Multicast load without failures.
- **TR-4**: Execute Multicast with random canFail crashes (verify no deadlocks).
- **TR-5**: Execute Multicast with Resurrection and Reinsertion.
- **TR-6**: Execute fully Dynamic Membership scenarios.
- **TR-7**: Run all above scenarios in *Burst Mode* (0ms messaging period) to stress concurrent queues.

16. Non-Functional Requirements

- **Performance**: Burst mode must not trigger memory leaks or queue overflow limits within the JRE.
- **Concurrency**: Thread-safe access must be guaranteed without using global locks that block independent actors.
- **Portability**: Must execute inside a standard Eclipse IDE without external dependencies.

Appendix A - Conceptual Interfaces

A.1 CTask

```java
package com.engine.multicast.core;
public interface CTask extends Runnable {
    int getLamportClock();
    void kill();
    void setCanFail(boolean canFail);
}
```

A.2 Actor

```java
package com.engine.multicast.core;
public interface Actor {
    String getId();
    void multicast(byte[] payload);
    void onDeliver(String senderId, long sequenceNumber, byte[] payload);
}
```

A.3 GroupListener

```java
package com.engine.multicast.membership;
public interface GroupListener {
    void onActorJoined(String actorId);
    void onActorLeft(String actorId);
    void onActorCrashed(String actorId);
}
```

A.4 Verifier

```java
package com.engine.multicast.validation;
public interface Verifier {
    void recordDelivery(String actorId, long messageId);
    void notifyCrash(String actorId);
    void notifyReinsertion(String actorId);
    void verifyTotalOrder();
}
```

Appendix B - Glossary

- **JRE**: Java Runtime Environment.
- **SPOF**: Single Point of Failure (eliminated in operational phase).
- **Burst Mode**: A testing configuration where the artificial message delay is reduced to 0ms.